Walkthrough completo — paso a paso

LegalDataPlatform

Humberto Henriquez — Senior Data Engineer / Solutions Architect

Generado 2026-04-24

Walkthrough completo — LegalDataPlatform

Este documento explica **qué se implementó, en qué orden, y por qué se tomó cada decisión**. Está diseñado para defender en una entrevista técnica y para que cualquier ingeniero nuevo al proyecto pueda entender el sistema en 30 minutos.

Índice

1. Contexto y problema
2. Arquitectura general: Medallion + Source of Truth
3. Stack tecnológico — decisiones y alternativas
4. Capa 0 — Configuración y observabilidad
5. Capa 1 — Base de datos PostgreSQL
6. Capa 2 — Extractores (ingesta)
7. Capa 3 — Transformers y calidad
8. Capa 4 — Loaders
9. Capa 5 — AWS (S3, Lambda, Glue, SQS)
10. Capa 6 — Orquestación con Prefect
11. Capa 7 — IaC con Terraform
12. Tests y benchmarks
13. Flujo de datos end-to-end
14. Cómo correr todo localmente
15. Decisiones arquitectónicas clave (ADRs)
16. Limitaciones conocidas y roadmap

---

1. Contexto y problema

El rol requiere tres capacidades centrales:

1. **Pipelines ETL/ELT escalables en Python y AWS** para ingerir datos legales y comerciales de múltiples fuentes.
2. **Optimización de PostgreSQL** para mantener datos relacionales complejos con alto rendimiento.
3. **Calidad y normalización de datos** para sostener el estándar de "Fuente de verdad".

El sistema debe:

- Ingerir datos de APIs REST, archivos CSV/Parquet, bases de datos externas y eventos S3.
- Normalizar y validar cada registro antes de tocar la capa productiva.

- Escribir a PostgreSQL con alto throughput.
- Mantener histórico (SCD2) de entidades que cambian en el tiempo.
- Ser auditable (lineage), reproducible (idempotente) y observable (logs/metrics).

2. Arquitectura general: Medallion + Source of Truth

Se eligió la **arquitectura Medallion** (Bronze/Silver/Gold) porque separa responsabilidades con claridad:

Capa	Contenido	Formato	Target
Bronze	Datos crudos tal cual llegaron del origen	Parquet + Snappy en S3	Inmutable, sirve como "safety net" para reproducir transformaciones
Silver	Datos normalizados y validados	Parquet en S3 + PostgreSQL	Datos limpios listos para consumo operativo
Gold	Agregados y vistas de negocio	PostgreSQL (MViews) + S3	BI, dashboards, ML features
Quarantine	Registros rechazados con el motivo	Parquet en S3	Triage manual, RCA

PostgreSQL es la "Fuente de Verdad" operativa porque:

- Transacciones ACID donde las necesitamos (contratos, facturación).
- JOINs complejos con datos relacionales.
- Constraints (FK, CHECK) que impiden estados inválidos.
- Materialized views para analytics pesado sin mover a otra DB.

S3 sirve como **archivo histórico y área de staging**. Cuando hay que re-procesar una ventana de 6 meses de datos, se lee del Bronze, no se vuelve a llamar a las APIs.

3. Stack tecnológico — decisiones y alternativas

Decisión	Escogido	Alternativas evaluadas	Razón
Lenguaje	Python 3.11	Scala, Java, Go	Productividad del equipo, ecosistema de datos. 3.11 por `asyncio.TaskGroup` y mejor performance que 3.10.
Orquestador	Prefect 2	Airflow, Dagster, Mage	DAGs en Python puro (no DSL), mejor DX para equipos pequeños, retries y observability out-of-the-box. Airflow sigue siendo mejor a escala 500+ DAGs.
Compute de datos	Pandas + Polars	Spark, DuckDB	Pandas por familiaridad; Polars para cargas >1M rows (10-30× más rápido, memoria acotada). Spark sólo para Glue/Redshift cuando no cabe en un proceso.
DB relacional	PostgreSQL 16	MySQL, SQL Server	Particionado nativo, JSONB, full-text search, MVCC maduro, extensiones (`pg_trgm` , `pg_stat_statements`).
Pool	PgBouncer	pgcat, built-in pool	PgBouncer es el estándar probado en producción; modo transaction pooling da ratio 100:1.
Storage	AWS S3	GCS, Azure Blob	El rol menciona AWS explícitamente. S3 = 11 nines, integra con todo el ecosistema AWS.
Serverless compute	Lambda	ECS Fargate, EKS	Lambda para triggers puntuales y procesamiento ligero; Fargate si la carga excede 15min/10GB.
IaC	Terraform	CDK, CloudFormation	Terraform es multi-cloud, tiene mejor state management, y la comunidad de módulos es mayor.
Validación	Pydantic v2	marshmallow, attrs	v2 es 5-50× más rápido (Rust core), type-safe, soporta JSON Schema.
DQ	Great Expectations + engine propio	Soda, dbt tests	GE para documentación y suites formales; engine propio (YAML) para gates rápidos in-pipeline.

4. Capa 0 — Configuración y observabilidad

Archivos:

- src/config.py
- src/observability/logging.py
- src/observability/metrics.py

`src/config.py`

Uso **Pydantic Settings** en lugar de `os.environ.get()` porque:

- Validación en tiempo de arranque (fail fast): si falta una env var o tiene tipo incorrecto, la app no arranca.
- Separación por dominio: `PostgresSettings`, `AwsSettings`, `S3Buckets`, etc. — cada bloque se puede reusar aisladamente.
- `@lru_cache` en `get_settings()` garantiza que todo el proceso ve la misma configuración.

Las credenciales se leen de variables de entorno (12-factor); en producción éstas vienen de Secrets Manager vía la tarea ECS/Lambda, no de un `.env`.

Logging estructurado con `structlog`

En lugar de logs tipo `INFO - Processing batch 5`, emitimos JSON:

```
{"event": "batch_loaded", "pipeline": "legal_ingestion", "run_id": "...", "rows": 1500,
"duration_ms": 820}
```

Esto habilita:

- Aggregation en CloudWatch/Datadog/ELK sin regex brittle.
- Correlación por `run_id` a través de todas las etapas.
- Consultas analíticas sobre los logs (ej. "p95 de duración del stage load").

`bind_pipeline_context()` fija pipeline y run_id una vez y todos los logs subsecuentes los heredan automáticamente.

Métricas Prometheus

Exponemos contadores y histogramas que siguen la metodología **RED + data-specific**:

- `ldp_records_extracted_total{source, pipeline}` — rate de ingesta
- `ldp_records_loaded_total{target, pipeline}` — rate de escritura
- `ldp_records_quarantined_total{pipeline, rule}` — calidad fallida
- `ldp_pipeline_duration_seconds{pipeline, stage}` — duración por etapa
- `ldp_pipeline_errors_total` — rate de errores
- `ldp_dq_checks_total{suite, result}` — DQ pass/fail

Para jobs batch usamos `push_to_gateway()` (Pushgateway) porque los procesos no viven lo suficiente para ser scrapeados.

5. Capa 1 — Base de datos PostgreSQL

Archivos principales:

- src/database/models/ — ORM models
- src/database/migrations/versions/0001_initial_schema.py

- src/database/migrations/versions/0002_partitions_and_mviews.py
- src/database/optimization/ — partitioning, MViews, index tuning
- src/database/session.py

Modelo de datos

El modelo cubre dos dominios principales:

Legal:
- `legal_entities` — empresas/personas referenciadas.
- `legal_documents` — sentencias, regulaciones, actas. **Particionada por** `document_date` **mensualmente**.
- `regulations` — normativas con rango de vigencia.

Commercial:
- `counterparties` — clientes/vendors.
- `contracts` — contratos comerciales.
- `transactions` — transacciones financieras. **Particionada por** `transaction_date` **mensualmente**.

Dimensiones:
- `dim_counterparty` — **SCD Type 2** sobre contrapartes.
- `dim_jurisdiction` — SCD Type 1 (lookup simple).

Staging:
- `stg_legal_document` y `stg_transaction` — tablas **UNLOGGED** (no WAL) para escritura rápida antes de merge.

Por qué particionar

Tablas de transacciones crecen ~1M rows/día en escala media. Sin particionado:

- Los índices BTree se vuelven multi-GB → queries lentos.
- `VACUUM` bloquea la tabla completa.
- Retention (`DELETE WHERE date < ...`) toma horas.

Con particionado por mes:

- El **planner** hace *partition pruning* automáticamente → sólo escanea particiones relevantes.
- `VACUUM` y `REINDEX` son por partición, no globales.
- Retention es `DROP PARTITION`, O(1) vs O(N).
- Parallel scans funcionan mejor.

El script src/database/optimization/partitioning.sql define una función auxiliar `create_monthly_partition()` y pre-crea 12 meses hacia adelante. Se debe correr mensualmente vía cron o usar `pg_partman` para automatizarlo.

Índices que importan

Cada índice cuesta write-amplification, por eso cada uno está justificado:

- **BTree compuesto** en `(counterparty_id, transaction_date DESC)` — soporta queries "últimas N transacciones por contraparte".

- **BRIN** en `transaction_date` — 10KB/millón de rows vs 30MB para BTree. Ideal para columnas naturalmente clusterizadas (inserts time-ordered).
- **Partial index** en contratos activos: `WHERE status = 'ACTIVE'` — reduce tamaño del índice 70%+.
- **GIN + pg_trgm** para búsqueda fuzzy sobre `legal_name` y `title` de documentos.
- **GIN jsonb_path_ops** sobre `metadata` JSONB — 3× más rápido que default `gin_default_ops` para queries de contención `@>`.
- **Covering index** con `INCLUDE (amount, currency)` — evita el lookup a heap para queries de agregación.

Ver src/database/optimization/indexes_additional.sql.

Materialized Views

`mv_monthly_revenue_per_counterparty` pre-computa agregados mensuales. Refresco con `CONCURRENTLY` para no bloquear lecturas.

Se llama desde Prefect:

```
CALL refresh_analytical_views();
```

Connection pooling

- **Direct engine** (`direct_session`) para migraciones, `COPY` bulk, transacciones largas.
- **PgBouncer engine** (`pooled_session`) para workloads concurrentes — pool en modo `transaction`, que da un ratio 100:1 de conexiones cliente a backend.

Importante: al usar PgBouncer en transaction mode con asyncpg, **se debe deshabilitar el statement cache** (`statement_cache_size=0`), porque PgBouncer rota backend connections entre transacciones.

Optimización continua

src/database/optimization/query_helpers.py provee:

- `explain_analyze()` → parsea el plan JSON y calcula cache hit ratio.
- `top_slow_queries()` → lee `pg_stat_statements` para detectar queries patológicos.
- `table_stats()` → devuelve tamaño, row estimate, bloat ratio, last vacuum.
- `vacuum_analyze()` → dispara VACUUM ANALYZE asíncrono.

6. Capa 2 — Extractores (ingesta)

Todos los extractores implementan un protocolo común:

```
class Extractor(ABC):
    async def extract(self) -> AsyncIterator[ExtractBatch]: ...
```

Cada batch incluye **metadata de lineage** (batch_id, source_system, source_name, extracted_at) que viaja con el dato hasta el destino final, permitiendo responder "¿de dónde vino esta fila?".

REST API (rest_api.py)

- Soporta paginación **offset / cursor / page**.
- Retries con `tenacity` (exponential backoff max 30s, 5 intentos).
- Respeta `Retry-After` header en 429.
- Extrae datos por JSON path (`data.items` etc.).
- Emite métricas por request.

CSV (csv_extractor.py)

Usa **Polars** `scan_csv` (lazy) para archivos que pueden no caber en RAM. Chunks configurables; el lazy scan hace pushdown de columnas si luego se selecciona.

DB-to-DB (db_extractor.py)

Patrón **watermark-based incremental**: se guarda el último `updated_at` procesado en tabla `pipeline_watermarks`, y cada run sólo trae filas `WHERE updated_at > :watermark`. Idempotente y resistente a interrupciones.

S3 Parquet (s3_extractor.py)

Lee row-group por row-group para mantener memoria acotada. Soporta column pruning (sólo lee las columnas que se pasan).

7. Capa 3 — Transformers y calidad

Pydantic como primera barrera

src/schemas/ define schemas Pydantic para cada entidad. Los validadores hacen **normalización + validación**:

- `legal_name` → mayúsculas, espacios colapsados.
- `tags` → lowercase, dedup, strip.
- `currency` → uppercase forzado (`to_upper=True`).
- `transaction_type` → uppercase.
- `status` → restringido a un set permitido.
- Rangos numéricos (risk_score 0-100, amount no-cero, total_value >= 0).
- Coherencia de fechas (end_date >= start_date).

Normalizadores (normalizers.py)

Procesan batches aplicando el schema y devuelven `NormalizationResult{valid, rejected}`. Los rechazos llevan el error detallado de Pydantic para que el triaje sea inmediato.

SCD2 (scd2.py)

El handler SCD2:

1. Calcula un `row_hash` sobre los campos tracked (detector de cambios eficiente).
2. Busca la versión actual (`is_current=true`) del natural key.
3. Si no existe → INSERT como actual.
4. Si existe y hash coincide → unchanged.
5. Si existe y hash distinto → expira la vieja (`is_current=false` , `valid_to = effective_date - 1`) e inserta la nueva.

Esto permite consultas point-in-time: "¿cuál era el risk_score de CP-123 el 2024-06-01?" vía `WHERE '2024-06-01' BETWEEN valid_from AND valid_to` .

Rule engine declarativo (rule_engine.py)

Reglas en YAML (legal_documents.yaml, transactions.yaml) con tipos:

- `not_null` , `unique` , `regex` , `in_set` , `range` , `row_count_range` , `expression` .
- `severity: error` bloquea el pipeline; `warning` sólo emite métrica.

Ejemplo:

```yaml
rules:
  - name: document_hash_is_sha256
    type: regex
    column: source_hash
    params: { pattern: "^[a-f0-9]{64}$" }
    severity: error
```

Great Expectations (ge_integration.py)

Complementa el rule engine con:

- **Data Docs** auto-generadas (HTML).
- Expectations estadísticas (medias, cuantiles, distribuciones).
- Data profiling para bootstrappear suites nuevas.

8. Capa 4 — Loaders

PostgreSQL bulk (postgres_bulk.py)

Usa el **protocolo COPY** nativo de Postgres vía asyncpg:

- INSERT individual: ~1,000 rows/s.
- COPY: ~50,000-200,000 rows/s (depende de índices y WAL).

Para UPSERT usamos el patrón **staging table**:

1. `CREATE TEMP TABLE stg (LIKE target)` on commit drop.

2. `TRUNCATE stg`.
3. `COPY stg FROM STDIN`.
4. `INSERT INTO target SELECT * FROM stg ON CONFLICT (...) DO UPDATE SET ...`.

Esto es 10-100× más rápido que row-by-row upserts y mantiene atomicidad.

S3 Parquet (s3_parquet.py)

Escribe a S3 particionado por `ingestion_date`:

```
s3://bronze/legal_documents/ingestion_date=2026-04-24/part-20260424T120000-0.parquet
```

Snappy compression por defecto — buen tradeoff velocidad/tamaño; **ZSTD** cuando el costo de almacenamiento importa más que el de CPU.

9. Capa 5 — AWS (S3, Lambda, Glue, SQS)

S3 client (s3_client.py)

Wrapper fino sobre aioboto3 que:

- Inyecta el endpoint de MinIO cuando `AWS_S3_ENDPOINT` está seteado.
- Aplica SSE-KMS por defecto.
- Pagina automáticamente `list_objects_v2`.

Lambda handlers

`s3_event_ingest` — gatillado por PUT en Bronze:

1. Recibe el evento S3.
2. Construye mensaje SQS con `{bucket, key, size, event_time}`.
3. Batch-envía a SQS (max 10 por batch).

`sqs_pipeline_trigger` — consume SQS:

1. Para cada mensaje, llama al Prefect API `/deployments/{id}/create_flow_run`.
2. Devuelve `batchItemFailures` para que SQS re-envíe sólo los mensajes fallidos (partial batch response).

Decoupling deliberado: Lambda no ejecuta el pipeline porque (a) timeout 15min, (b) memoria 10GB, (c) paquete muy grande. Delega a ECS/Fargate/worker Prefect.

Glue PySpark (legal_doc_normalize.py)

Ruta de escala para cuando el volumen supera lo que un proceso Python maneja. Normaliza Bronze→Silver para documentos legales: uppercase/trim, hash determinístico, dedup por hash, particionado de salida por `document_date`.

10. Capa 6 — Orquestación con Prefect

Flow legal (legal_ingestion_flow.py)

Secuencia:

1. **Extract** → CSV → batches.
2. **Bronze persistence** → escribe raw a S3 como Parquet.
3. **Normalize** → Pydantic + rule engine; errores → quarantine.
4. **Load** → `upsert_via_staging` a `legal_documents`.
5. **Gold refresh** → `CALL refresh_analytical_views()`.

Cada stage está envuelto en `pipeline_duration.labels( ... ).time()` para medir duración. Todos los tasks son `@task`-decorated con retries configurables.

Flow commercial (commercial_ingestion_flow.py)

Similar, pero con:

- Upsert de counterparties en la tabla operativa.
- **Aplicación de SCD2** sobre `dim_counterparty` para preservar histórico.
- Resolución de FKs (`contract_id`, `counterparty_id`) por lookup antes de cargar transacciones.

Deployments (deployments.py)

Registra los flows en Prefect con cron schedules:

- `legal-ingestion-daily` → 02:00 UTC diario.
- `commercial-ingestion-hourly` → minuto 15 de cada hora.

11. Capa 7 — IaC con Terraform

infra/terraform/ provisiona:

- **S3**: 4 buckets (bronze/silver/gold/quarantine), versioning, SSE-KMS, lifecycle policies (IA a los 30d, Glacier IR a los 90d).
- **SQS + DLQ**: cola `ingest` con redrive policy (5 attempts → DLQ), alarma CloudWatch al DLQ.
- **Lambda**: `s3_event_ingest` y `sqs_pipeline_trigger` con IAM roles scoped.
- **Event notification**: S3 PUT → SQS (filtro `.parquet`).
- **Glue**: catálogos legal/commercial, IAM role, upload de scripts PySpark, job `legal_doc_normalize`.
- **Aurora PostgreSQL Serverless v2**: cluster con scaling 0.5-16 ACU, Secrets Manager para credenciales, logs a CloudWatch.

Backend del state en S3 + DynamoDB para locking.

12. Tests y benchmarks

Tests unitarios (`tests/unit/`)

- `test_schemas.py` — validación Pydantic (happy path y errores).
- `test_normalizers.py` — valid/rejected counts.
- `test_rule_engine.py` — cada tipo de regla.
- `test_enrichers.py` — row hash determinismo, risk tier buckets.

Tests de integración (`tests/integration/`)

- `test_s3_handler.py` — contra moto (S3 mockeado en memoria).
- (Se pueden agregar tests con Testcontainers-Postgres para validar COPY real.)

Benchmarks (scripts/benchmarks/query_benchmark.py)

Mide p50/p95/p99 de queries representativas antes/después de optimizaciones:

- `count_transactions_last_30d` — usa partition pruning.
- `top_counterparties_by_revenue` — usa el covering index.
- `fuzzy_search_entity_name` — usa pg_trgm.
- `mv_monthly_revenue_scan` — valida que la MV da sub-ms.

13. Flujo de datos end-to-end

Escenario: llega un archivo CSV al SFTP de un regulador a las 01:45 UTC.

```
01:45  SFTP receives file → cron rsync a S3://ldp-incoming/{date}/file.csv
01:50  S3 PUT event → Lambda s3_event_ingest
01:50  Lambda → SendMessageBatch → SQS ingest queue
01:51  Lambda sqs_pipeline_trigger drena SQS → Prefect create_flow_run
01:52  Prefect dispara legal_ingestion_flow
       ├─ extract_from_csv (Polars scan, chunks de 5000)
       ├─ persist_to_bronze → S3 Parquet particionado
       ├─ normalize_and_validate → Pydantic + YAML rules
       │    └─ rechazos → S3 quarantine
       ├─ load_legal_documents → COPY staging + UPSERT
       └─ refresh_gold → REFRESH MATERIALIZED VIEW CONCURRENTLY
02:10  Grafana dashboard refleja el run via métricas Prometheus
02:11  Alertas si ldp_records_quarantined_total > threshold
```

14. Cómo correr todo localmente

```
# 1. Variables
cp .env.example .env

# 2. Stack
docker compose up -d
# → Postgres 16, PgBouncer, MinIO (S3 local), Prefect UI, Prometheus, Grafana

# 3. Dependencias
pip install -e ".[dev]"

# 4. Migraciones
alembic upgrade head

# 5. Datos de ejemplo
python scripts/seed_data.py
# → genera 5000 docs, 500 counterparties, 1500 contratos, 20000 txns

# 6. Pipeline
make pipeline
# o: python -m src.pipelines.orchestration.legal_ingestion_flow

# 7. UIs
# Prefect:    http://localhost:4200
# MinIO:      http://localhost:9001  (user: minioadmin / pass: minioadmin123)
# Prometheus: http://localhost:9090
# Grafana:    http://localhost:3000  (admin / admin)

# 8. Benchmarks
make benchmark

# 9. Tests
make test
```

15. Decisiones arquitectónicas clave (ADRs)

ADR-001: Particionado por RANGE(fecha) mensual

Contexto: tablas de transacciones y documentos crecen linealmente en el tiempo; queries de negocio casi siempre filtran por fecha.

Decisión: particionar ambas tablas por `document_date` / `transaction_date` con granularidad mensual.

Consecuencias positivas: pruning automático, VACUUM por partición, retention como DROP PARTITION.

Consecuencias negativas: FK compuestas (`(id, date)`) son más verbosas; UPSERT requiere que el conflict key incluya la partition key.

Alternativa considerada: particionar por HASH(id). Rechazado porque no habilita pruning por rango de fechas.

ADR-002: Medallion + S3 como safety net

Contexto: necesitamos poder re-procesar meses de datos sin volver a llamar APIs externas (rate limits, costo, inmutabilidad).

Decisión: Bronze en S3 almacena raw input; Silver normaliza; Gold agrega. PostgreSQL es fuente de verdad operativa pero S3 Bronze es la verdad histórica.

ADR-003: Prefect en lugar de Airflow

Contexto: equipo de 3-5 ingenieros, 10-30 flows proyectados.

Decisión: Prefect 2 por DX superior (Python puro, no DSL), retries nativos, mejor modelo de deployments.

Consecuencias: si crecemos >200 DAGs, revisitar — Airflow tiene mejor ecosistema de providers a esa escala.

ADR-004: Pydantic en lugar de dataclasses + validators manuales

Contexto: cada fuente trae datos "sucios"; necesitamos un solo punto de validación/normalización.

Decisión: Pydantic v2 schemas por entidad. Los validators hacen normalización (no sólo rechazo) para que la semántica de "canónico" esté centralizada.

ADR-005: COPY vs INSERT para bulk

Contexto: cargas de 10K-1M rows por batch.

Decisión: COPY vía asyncpg + UPSERT desde temp staging.

Benchmarks: INSERT ~1K rows/s vs COPY ~150K rows/s en hardware comparable → 100-150×.

ADR-006: SCD Type 2 sobre counterparties, Type 1 sobre jurisdictions

Contexto: counterparties tienen atributos (risk_score, name, tax_id) que cambian con el tiempo y el histórico importa para compliance. Jurisdictions son un lookup semi-estático.

Decisión: SCD2 para counterparties (histórico completo, `valid_from/valid_to/is_current`), SCD1 para jurisdictions (overwrite).

16. Limitaciones conocidas y roadmap

Limitaciones actuales

- El Glue job sólo cubre `legal_documents`; el commercial sigue en el path Python.
- No hay streaming CDC — los cambios upstream se detectan por watermark/polling, no por log-based CDC.
- Great Expectations está integrado pero no se expone como Data Docs HTML servidos.
- No se ha implementado un Data Catalog propio (se delega a Glue Catalog).
- PgBouncer corre en docker-compose, no es HA.

Roadmap (H3+)

1. **Streaming CDC con Debezium** → Kafka → Flink/ksqlDB → Silver near-real-time.
2. **Redshift Spectrum** sobre Silver para analytics cross-source.
3. **ML features store** (Feast) sobre Gold para modelos de risk scoring.
4. **Column-level lineage** (OpenLineage + Marquez).
5. **Data contracts** con schema registry (Confluent / Buf) para que productores y consumidores negocien schemas explícitamente.
6. **Cost observability** — tagging FinOps en S3/Glue/RDS para attribution por pipeline.

Fin del walkthrough.